

Clase 052 — HMAC y autenticación de mensajes

Parte: 2 — **Criptografía aplicada** · Fuente: *Cryptography Engineering* (Ferguson/Schneier/Kohno) e IETF RFC 2104 ·  Duración estimada: **90 min** · Nivel: **Intermedio**

Objetivo

Entender qué es un código de autenticación de mensajes (MAC), por qué la integridad sin autenticación no basta, y cómo HMAC combina una función hash con una clave secreta para garantizar que un mensaje proviene de quien dice y no fue alterado. El alumno aprenderá también el orden correcto de combinar cifrado y MAC (Encrypt-then-MAC) y por qué la comparación debe ser en tiempo constante.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Distinguir** integridad de autenticación e identificar qué aporta un MAC.
- 2. **Explicar** la construcción HMAC y por qué neutraliza la extensión de longitud.
- 3. **Generar y verificar** HMAC con OpenSSL y Python.
- 4. **Aplicar** correctamente el patrón Encrypt-then-MAC.
- 5. **Implementar** comparación en tiempo constante para evitar timing.

Temas

#	Tema	Por qué importa
1	MAC: definición y objetivo	Autenticidad de mensajes
2	Construcción HMAC	Estándar seguro y simple
3	HMAC vs hash con clave concatenada	Evita extensión de longitud
4	Encrypt-then-MAC vs MAC-then-Encrypt	Orden correcto
5	Comparación en tiempo constante	Evita fugas de timing
6	Claves y rotación	Gestión práctica
7	HMAC en la práctica (JWT, APIs)	Uso real

Definiciones y características

- **MAC**: etiqueta que autentica un mensaje usando una clave secreta compartida. Característica: sin la clave no se puede falsificar.
- **HMAC**: $HMAC(k, m) = H((k \oplus opad) || H((k \oplus ipad) || m))$. Seguro con cualquier hash decente, inmune a extensión de longitud.
- **Integridad vs autenticación**: un hash detecta cambios accidentales; un MAC detecta cambios maliciosos porque el atacante no tiene la clave.

- **Encrypt-then-MAC:** cifrar y luego calcular el MAC sobre el texto cifrado. Patrón recomendado; permite rechazar sin descifrar.
- **Comparación en tiempo constante:** comparar etiquetas sin salir antes en el primer byte distinto, evitando fugas temporales.
- **Clave simétrica de MAC:** debe ser distinta de la clave de cifrado y protegerse igual de bien.

Herramientas y preparación

```
openssl version
pip install cryptography
```

Laboratorio local. Las claves de HMAC son de práctica.

Laboratorio guiado

1. HMAC con OpenSSL:

```
bash echo -n "mensaje" | openssl dgst -sha256 -hmac "clave-secreta"
```

2. HMAC en Python y verificación segura:

```
python import hmac, hashlib key = b"clave-secreta" tag = hmac.new(key, b"mensaje",
hashlib.sha256).hexdigest() # verificación en tiempo constante ok = hmac.compare_digest(tag,
tag) print(tag, ok)
```

3. **Detecta manipulación:** envía `(mensaje, tag)`; el receptor recalcula el HMAC. Cambia un carácter del mensaje y comprueba que la verificación falla.
4. **Encrypt-then-MAC:** cifra con AES-CBC, calcula HMAC-SHA256 sobre `IV || ciphertext`, y verifica el MAC **antes** de descifrar. Documenta por qué esto previene padding oracle (clase 060).
5. **Timing (concepto):** implementa una comparación ingenua byte a byte con salida temprana y razona por qué filtra información; sustitúyela por `compare_digest`.

Ejercicios

1. Explica por qué `hash(clave || mensaje)` es inseguro y HMAC no.
2. Implementa Encrypt-then-MAC y verifica el MAC antes de descifrar.
3. ¿Por qué se usan claves distintas para cifrar y para el MAC?
4. Investiga cómo firma un JWT HS256 y verifica uno con tu HMAC.
5. Demuestra que cambiar un bit del mensaje invalida el tag.
6. Compara HMAC con los MAC integrados en AEAD (GCM/Poly1305).

Reto verificable

Implementa un canal autenticado sencillo: el emisor envía `(IV, ciphertext, HMAC)` y el receptor rechaza cualquier mensaje manipulado sin descifrarlo. **Criterio de aceptación:** alterar cualquier byte del IV, del texto cifrado o del tag provoca rechazo, y solo los mensajes íntegros se descifran.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
MAC-then-Encrypt con CBC	Habilita padding oracle; usa Encrypt-then-MAC o AEAD
Comparar tags con <code>==</code>	Fuga de timing; usa <code>compare_digest</code>
Reutilizar la clave de cifrado como clave de MAC	Debilita ambas; deriva claves separadas
Verificar el MAC después de descifrar	Ya procesaste datos no autenticados; verifica antes
Hash simple como "firma" de API	No autentica origen; usa HMAC con clave secreta

? Preguntas frecuentes

? **¿HMAC o AEAD?** Si ya cifras, prefiere AEAD (GCM, ChaCha20-Poly1305), que integra el MAC. HMAC brilla para autenticar datos que no ciframos (tokens, webhooks).

? **¿Puedo usar HMAC-MD5?** HMAC-MD5 resiste mejor que MD5 solo, pero usa HMAC-SHA256 por buenas prácticas.

? **¿El MAC da confidencialidad?** No; solo autenticidad e integridad. Combínalo con cifrado para confidencialidad.

🔗 Referencias

- IETF RFC 2104 (HMAC) — <https://www.rfc-editor.org/rfc/rfc2104>
- Ferguson, Schneier, Kohno, *Cryptography Engineering*, cap. 6–7.
- NIST FIPS 198-1 (HMAC) — <https://csrc.nist.gov/publications/detail/fips/198/1/final>
- OWASP, "Cryptographic Storage Cheat Sheet".

➡ Siguiente clase

Clase 053 - Intercambio de claves: Diffie-Hellman